



山东大学

信息科学与工程学院

2024—2025 学年第二学期

实 验 报 告

课程名称： 数据结构与算法导论

实验名称： 散列查找的编程与实现

一、实验目的

掌握散列查找的基本概念及其基本操作的实现

二、实验原理

参照课本 p.111-123, 及 Figure5.3-5.17

散列查找是一种通过关键字直接计算存储地址的查找方法。它的基本思想是：根据给定的关键字 key , 通过哈希函数 $H(key)$ 计算出该元素在哈希表中的存储位置, 从而减少查找过程中逐个比较的次数, 提高查找效率。本实验要求构造哈希函数并实现哈希查找, 同时需要创建哈希空表、处理冲突并完成查找操作。

本实验采用除留余数法构造哈希函数, 其形式为:

$$H(key) = key \bmod m$$

其中, key 表示待插入或待查找的关键字, m 表示哈希表的长度。通过该函数可以把关键字映射到哈希表中的某一个下标位置。

由于不同关键字可能通过哈希函数得到相同的地址, 因此会产生**哈希冲突**。为了解决冲突, 本实验采用开放定址法。开放定址法的基本思想是: 当计算得到的位置已经被占用时, 按照一定的探测规则继续寻找下一个空位置。

本程序中提供了两种冲突处理方法:

线性探测法:

$$H_i = (H_0 + i) \bmod m$$

当发生冲突时, 依次向后查找下一个位置, 直到找到空位置或查找成功。

二次探测法:

$$H_i = (H_0 \pm i^2) \bmod m$$

当发生冲突时, 不是连续向后查找, 而是按照平方增量进行跳跃探测, 从而减少线性探测中容易出现的连续堆积现象。

在查找关键字时, 首先通过哈希函数计算其初始地址。如果该位置的元素正好等于待查找关键字, 则查找成功; 如果该位置为空, 则说明查找失败; 如果该位置已被其他关键字占用, 则按照插入时相同的冲突处理方法继续探测, 直到找到目标元素或确定该元素不存在。这样可以保证插入和查找过程的一致性。

三、实验内容

- 1) 创建一个 hash 空表, 实现动态空间分配的初始化
- 2) 构造一个 hash 函数和某种处理冲突的方法将数据元素存放到 hash 表中
- 3) 设计相应的 hash 查找函数对 hash 表中的元素进行查找
- 4) hash 构造函数可选用除留余数法或其他方法
- 5) 处理冲突可选开放定址法或其他方法

四、实验要求

- 1) 构造哈希函数, 实现哈希查找

五、实验源程序

```

#include <stdio.h>
#include <stdlib.h>

#define EMPTY -1
#define DELETED -2

typedef int ElemType;

typedef struct {
    ElemType key;
} HashNode;

typedef struct {
    HashNode *data;    // 动态分配的哈希表空间
    int tableSize;     // 哈希表长度
    int count;         // 当前已存入元素个数
    int method;        // 冲突处理方法: 1 线性探测, 2 二次探测
} HashTable;

/* 哈希函数: 除留余数法 */
int Hash(HashTable *ht, ElemType key) {
    return key % ht->tableSize;
}

/* 初始化哈希表 */
void InitHashTable(HashTable *ht, int size, int method) {
    int i;

    ht->tableSize = size;
    ht->count = 0;
    ht->method = method;

    ht->data = (HashNode *)malloc(sizeof(HashNode) * size);
    if (ht->data == NULL) {
        printf("内存分配失败! \n");
        exit(1);
    }

    for (i = 0; i < size; i++) {
        ht->data[i].key = EMPTY;
    }
}

/* 根据冲突处理方式计算下一次探测位置 */

```

```

int GetNextPos(HashTable *ht, int start, int k) {
    int pos;

    if (ht->method == 1) {
        // 线性探测:  $H_i = (H_0 + i) \% m$ 
        pos = (start + k) % ht->tableSize;
    } else {
        // 二次探测:  $H_i = (H_0 \pm i^2) \% m$ 
        if (k % 2 == 1) {
            pos = (start + (k + 1) / 2 * ((k + 1) / 2)) % ht->tableSize;
        } else {
            pos = (start - (k / 2) * (k / 2)) % ht->tableSize;
            if (pos < 0) {
                pos += ht->tableSize;
            }
        }
    }

    return pos;
}

/* 插入元素 */
int InsertHash(HashTable *ht, ElemType key) {
    int start, pos, i;

    if (ht->count >= ht->tableSize) {
        printf("哈希表已满, 无法插入 %d! \n", key);
        return 0;
    }

    start = Hash(ht, key);
    pos = start;

    for (i = 0; i < ht->tableSize; i++) {
        if (ht->data[pos].key == EMPTY) {
            ht->data[pos].key = key;
            ht->count++;
            return 1;
        } else if (ht->data[pos].key == key) {
            printf("元素 %d 已存在, 不重复插入。 \n", key);
            return 0;
        } else {
            pos = GetNextPos(ht, start, i + 1);
        }
    }
}
    
```

```

    }

    printf("插入 %d 失败, 冲突无法解决。\\n", key);
    return 0;
}

/* 创建哈希表 */
void CreateHashTable(HashTable *ht) {
    int size, n, method;
    int i, key;

    printf("请输入哈希表长度: ");
    scanf("%d", &size);

    printf("请输入待插入元素个数: ");
    scanf("%d", &n);

    printf("请选择冲突处理方法: 1.线性探测 2.二次探测: ");
    scanf("%d", &method);

    if (method != 1 && method != 2) {
        printf("输入错误, 默认采用线性探测法。\\n");
        method = 1;
    }

    InitHashTable(ht, size, method);

    printf("请依次输入 %d 个关键字: \\n", n);
    for (i = 0; i < n; i++) {
        scanf("%d", &key);
        InsertHash(ht, key);
    }
}

/* 哈希查找 */
int SearchHash(HashTable *ht, ElemType key) {
    int start, pos, i;

    start = Hash(ht, key);
    pos = start;

    for (i = 0; i < ht->tableSize; i++) {
        if (ht->data[pos].key == EMPTY) {
            return -1;
        }
    }
}

```

```

    }

    if (ht->data[pos].key == key) {
        return pos;
    }

    pos = GetNextPos(ht, start, i + 1);
}

return -1;
}

/* 输出哈希表 */
void PrintHashTable(HashTable *ht) {
    int i;

    printf("\n 当前哈希表如下: \n");
    printf("下标: ");
    for (i = 0; i < ht->tableSize; i++) {
        printf("%4d", i);
    }

    printf("\n 数据: ");
    for (i = 0; i < ht->tableSize; i++) {
        if (ht->data[i].key == EMPTY) {
            printf("%4s", "-");
        } else {
            printf("%4d", ht->data[i].key);
        }
    }

    printf("\n");
}

/* 释放空间 */
void DestroyHashTable(HashTable *ht) {
    if (ht->data != NULL) {
        free(ht->data);
        ht->data = NULL;
    }
    ht->tableSize = 0;
    ht->count = 0;
}

```

```
int main() {
    HashTable ht;
    int key, pos;

    CreateHashTable(&ht);
    PrintHashTable(&ht);

    while (1) {
        printf("\n 请输入要查找的关键字，输入 -1 结束: ");
        scanf("%d", &key);

        if (key == -1) {
            break;
        }

        pos = SearchHash(&ht, key);

        if (pos == -1) {
            printf("查找失败，关键字 %d 不在哈希表中。\n", key);
        } else {
            printf("查找成功，关键字 %d 的位置是 %d。 \n", key, pos);
        }
    }

    DestroyHashTable(&ht);

    return 0;
}
}
```

六、实验结果

分别使用线性探测和二次探测再散列处理冲突之后进行查找

```

● PS D:\TEX\IEEE> cd "d:\Epan\Study\数据结构\源程序\" ; if ($?) { gcc Shiyang5.c -o Shiyang5
请输入哈希表长度: 7
请输入待插入元素个数: 4
请选择冲突处理方法: 1.线性探测 2.二次探测: 1
请依次输入 4 个关键字:
8 9 4 5

当前哈希表如下:
下标:    0    1    2    3    4    5    6
数据:    -    8    9    -    4    5    -

请输入要查找的关键字, 输入 -1 结束: 9
查找成功, 关键字 9 的位置是 2。

请输入要查找的关键字, 输入 -1 结束: 3
查找失败, 关键字 3 不在哈希表中。

请输入要查找的关键字, 输入 -1 结束: -1
○ PS D:\Epan\Study\数据结构\源程序> cd "d:\Epan\Study\数据结构\源程序\" ; if ($?) { gcc Shiyang6.c -o Shiyang6
请输入哈希表长度: 9
请输入待插入元素个数: 6
请选择冲突处理方法: 1.线性探测 2.二次探测: 2
请依次输入 6 个关键字:
12 4 9 10 15 7

当前哈希表如下:
下标:    0    1    2    3    4    5    6    7    8
数据:    9   10   -   12    4   -   15    7   -

请输入要查找的关键字, 输入 -1 结束: 2
查找失败, 关键字 2 不在哈希表中。

请输入要查找的关键字, 输入 -1 结束: 10
查找成功, 关键字 10 的位置是 1。

请输入要查找的关键字, 输入 -1 结束:

```

七、实验心得

在本次实验中,我完成了散列表的创建、插入、冲突处理和查找等基本操作,对散列查找的基本思想有了更清晰的理解。散列查找的核心是通过哈希函数将关键字映射到表中的某个位置,从而减少顺序比较的次数,提高查找效率。本实验中,我采用除留余数法构造哈希函数,即通过关键字对哈希表长度取余来确定初始存储位置,并在此基础上实现了元素的插入和查找操作,符合实验中“构造哈希函数,实现哈希查找”的要求。

在实验过程中,我重点理解了哈希冲突的处理方法。当两个不同关键字计算出的哈希地址相同时,就会产生冲突。为了解决这一问题,程序中采用了开放定址法,并分别实现了线性探测和二次探测。通过运行测试可以发现,当没有冲突时,元素会直接存放到哈希函数计算出的下标位置;当发生冲突时,线性探测会按顺序向后寻找空位置,而二次探测则按照平方增量进行跳跃查找。通过对比两种方法,我认识到线性探测实现简单,但容易产生元素连续堆积;二次探测可以在一定程度上减少堆积现象,但探测过程相对复杂。

通过本次实验,我不仅掌握了散列表的基本存储结构和查找过程,也加深了

对哈希函数、冲突处理和查找效率之间关系的理解。在编写和调试程序的过程中，我进一步熟悉了动态空间分配、结构体设计以及函数模块化的使用方法。总体来看，本次实验帮助我把课本中的散列查找理论转化为了具体的程序实现，提高了我分析和解决数据结构问题的能力。